Class Connect Web Application

Project Overview

🔷 **Project Name:** ClassConnect

🔷 **Description:**
ClassConnect is a web-based education portal designed to manage teachers, students, and administrative functionalities efficiently. It allows administrators to handle teacher registrations, student registrations, file uploads for profile images, and automated email notifications in a secure environment.

🔷 **Technologies Used:**

- **Backend:** Spring Boot, Spring Security, JPA/Hibernate
- **Frontend:** HTML, CSS, Thymeleaf
- **Database:** MySQL
- **Build Tool:** Maven
- **Server Port:** 8080
- **Deployment URL:** `http://class.connect.edu.in:8080/`

🔷 **Key Modules:**

1. **Admin Login & Dashboard**
2. **Teacher Management** (Add, Update, Delete, View)
3. **Student Management** (Add, Update, Delete, View)
4. **Add Student Through Teacher**
5. **File Upload Management** (Profile images)
6. **Email Notifications** (Registration, Updates)
7. **Session Management & Security**

🔷 **Purpose:**
To streamline class administration by providing a centralised platform for managing all educational data, improving efficiency, and reducing manual workload.

Spring Security Module:

The Spring Security module in ClassConnect is designed to provide robust authentication and authorization mechanisms for Admin, Teacher, and Student panels. This security configuration is defined within the SecurityConfig class, which uses multiple SecurityFilterChains to segregate security rules per user type. Each panel has custom login success handlers (CustomSuccessHandler, CustomSuccessHandlerAdmin, CustomSuccessHandlerStudent) to manage post-login session data, and authentication failure handlers to redirect users to their respective login pages with error messages upon failed login attempts.

The module employs role-based access controls, restricting endpoints (/admin/**, /teachers/**, /students/**) based on authenticated user roles. It uses DaoAuthenticationProvider beans for each user type with BCrypt password encoding for secure credential management. Remember-Me functionality is implemented with custom JDBC token repositories (CustomJdbcTokenRepositoryImpl), storing tokens in separate tables for each user group, ensuring persistent login sessions.

Additionally, custom logout handlers are integrated to update logout timestamps in the database and invalidate sessions securely. The configuration integrates JWT filtering for teachers, CSRF protection is disabled as per application design, and session management is configured to create sessions only when required. Global @ControllerAdvice classes populate user data globally for views based on the session, enhancing UI personalization. This modular and segregated security architecture ensures high maintainability, clarity, and granular access control across the multi-user educational portal.

API Package Documentation – JWT Module

The API package implements the JWT-based authentication system for secure REST API endpoints in the ClassConnect application. It includes:

ApiResponse.java – A simple response model containing success (boolean) and message (String) fields to standardize API responses for client consumption.

JWTService.java – A service class handling JWT operations:

Generates secure tokens dynamically using HmacSHA256 keys.

Encodes and decodes tokens via io.jsonwebtoken library.

Sets token claims such as subject (username), issued date, and expiration (configurable validity).

Validates tokens by verifying the signature and ensuring they are not expired.

Extracts username or specific claims from provided tokens for authentication context setup.

JwtFilter.java – A Spring OncePerRequestFilter that:

Intercepts incoming HTTP requests.

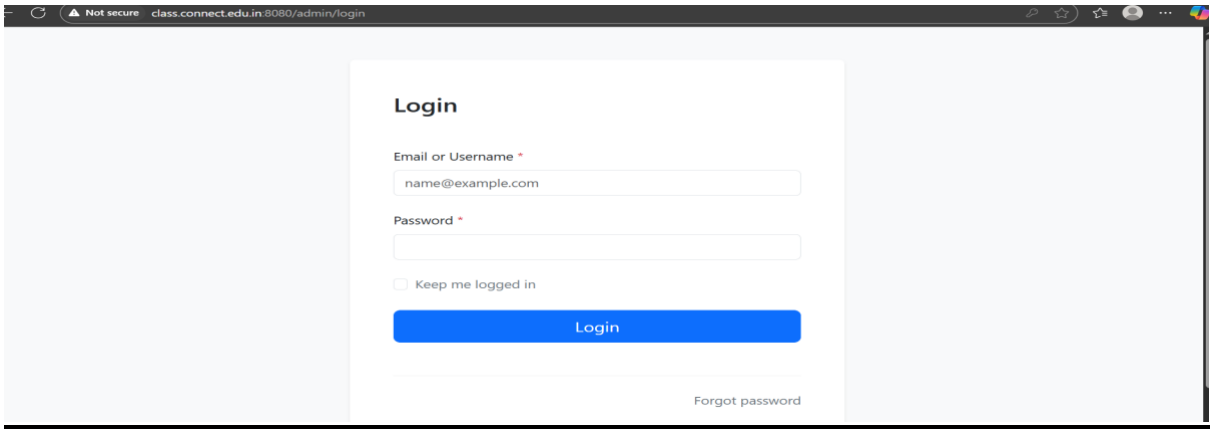
Checks for the Authorization header with a Bearer token.

Validates the JWT using JWTService.

Sets up the Spring SecurityContext with an authenticated user if the token is valid, enabling secure access to protected endpoints without session-based login.

This JWT module ensures that APIs remain stateless, secure, and scalable, enabling authenticated requests for teacher modules and potentially other user types in future extensions.

Admin Login Panel



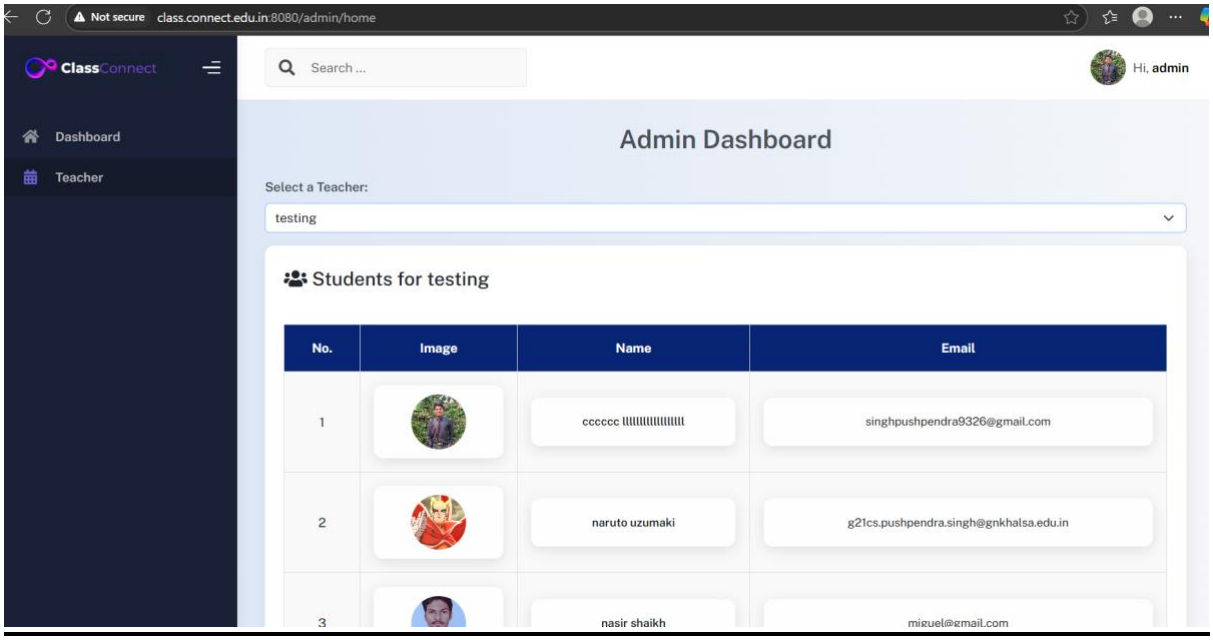
The **Admin Login module** in ClassConnect allows administrators to securely log in to access the admin dashboard and manage system functionalities. The frontend login page is accessible at `/admin/login`, where the admin enters their username or email along with the password. The page also provides a "Keep me logged in" checkbox to maintain the session and a "Forgot password" link for password recovery.

On the backend, this module uses **Spring Security** for authentication. The `AdminLoginService` class implements `UserDetailsService`, which is required by Spring Security to load user details during login. In the `loadUserByUsername` method, the service searches for an admin user in the database by username or email. If a matching user is found, it returns a `User` object with the username, password, and role set as "ADMIN". This allows Spring Security to authenticate and authorise the admin user based on these credentials.

Additionally, after successful login, the service uses the `storeUserDataInSession` method to store important admin details in the HTTP session, such as username, email, adminId, and sessionId. It also updates the login timestamp and saves it back to the database to keep track of login activity. The `AdminLoginRepo` interface, extending `JpaRepository`, handles all database operations for admin login, providing methods to find users by username or email.

Overall, this module ensures **secure authentication, role-based access, and session management** for admin users using Spring Security, making it a robust and essential component of the ClassConnect system.

Admin Dashboard Module



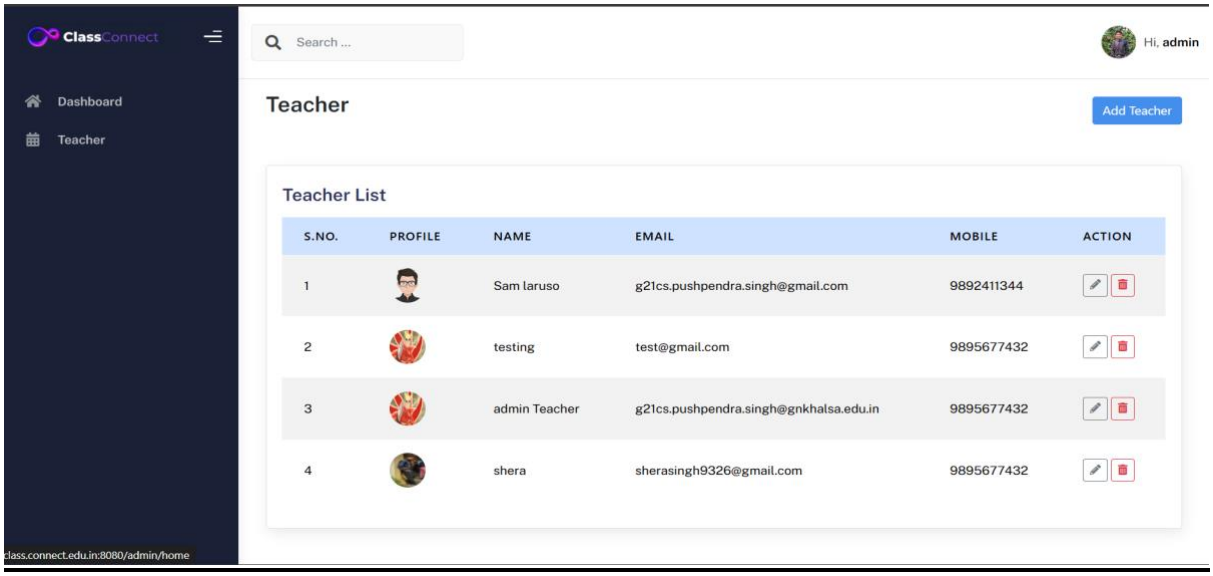
The **Admin Dashboard** page in ClassConnect displays a dropdown containing the list of teachers. When an admin selects any teacher from the dropdown, the dashboard dynamically shows the list of students registered under that particular teacher. The page is accessible at `/admin/home` after successful admin login.

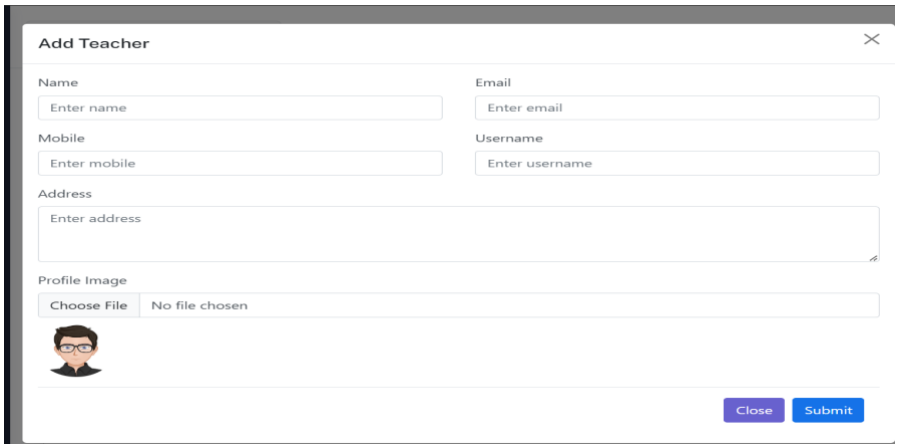
On the backend, this functionality is managed by the `AdminDashboardController` class within the `adminController` package. It has two main endpoints. The first endpoint, mapped to `admin/home`, is used to load the dashboard page. In this method, it calls the `AdminTeacherService` to fetch all teachers as an ArrayList of `AdminTeacherModel` objects and adds them to the model attribute `teachers`. This data is used to populate the teacher dropdown on the dashboard page.

The second endpoint is mapped to `admin/home/{id}` and is annotated with `@ResponseBody`, making it return data directly as JSON. When a teacher is selected on the frontend, an AJAX call is made to this endpoint with the teacher's ID. The controller then calls `AdminDashboardService` to fetch the list of students associated with the selected teacher ID, returning an ArrayList of `StudentLoginModel` objects as the response. This data is then displayed in a table format on the dashboard, showing student images, names, and emails for that teacher.

Overall, the Admin Dashboard module uses **Spring MVC controllers, model attributes, and AJAX JSON responses** to provide a dynamic and user-friendly interface for viewing teacher-wise student data efficiently.

Admin Teacher Module:



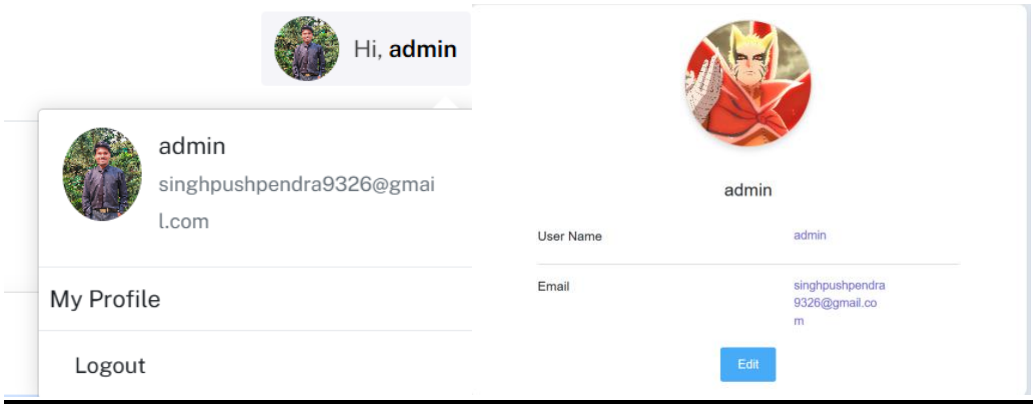


The **Teacher Module** allows the admin to view, add, update, and delete teacher records. On the Teacher page, a table displays the list of all teachers with their profile images, names, emails, and mobile numbers, along with edit and delete action buttons. An **"Add Teacher" button** opens a form popup where the admin can enter teacher details, including name, email, username, phone number, address, and profile image.

The backend is handled by the `AddTeacherController` class. The **`/admin/addTeacher` POST endpoint** manages both adding and updating teachers. If a `teacher_id` is provided, it updates the teacher record; otherwise, it adds a new teacher. It calls the `AddTeacherService` for both operations and handles file upload for teacher profile images using `MultipartFile`.

The **`/admin/deleteTeacher/{teacher_id}` GET endpoint** deletes a teacher by their ID using the `AddTeacherService` and then redirects back to the teacher list page. This module ensures efficient teacher management by providing CRUD operations in a simple and user-friendly interface.

Admin Profile Module:

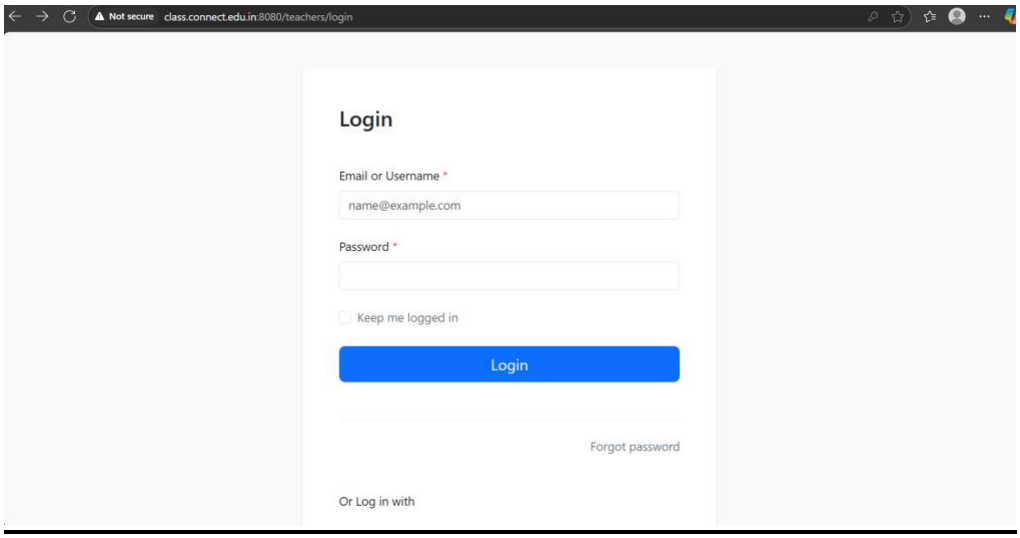


The Admin Profile module allows the admin to view and update their profile details, including username, email, and profile image. On the frontend, the profile dropdown in the navbar displays the admin's name, email, and image with links for "My Profile" and "Logout." Clicking "My Profile" navigates to the profile view page, which shows the admin's profile image, username, and email with an option to edit.

The backend is managed by the AdminProfController class. The /admin/profile GET endpoint displays the profile view page by fetching admin data from the database using the AdminLoginRepo based on the adminId stored in the session. The /admin/editProfile GET endpoint shows the editable form populated with existing details for editing. The /admin/updateProfile POST endpoint handles profile updates, including file upload for the profile image. If an image file is provided, it saves the image to a designated folder under uploads/images/admin/{adminId}/ and updates the image URL in the database; otherwise, it retains the existing image. After updating, it redirects to the profile page to display updated details.

This module ensures simple profile management for the admin, including secure updates with image handling, enhancing the user experience and personalisation of the admin account in ClassConnect.

Teacher Login Module:

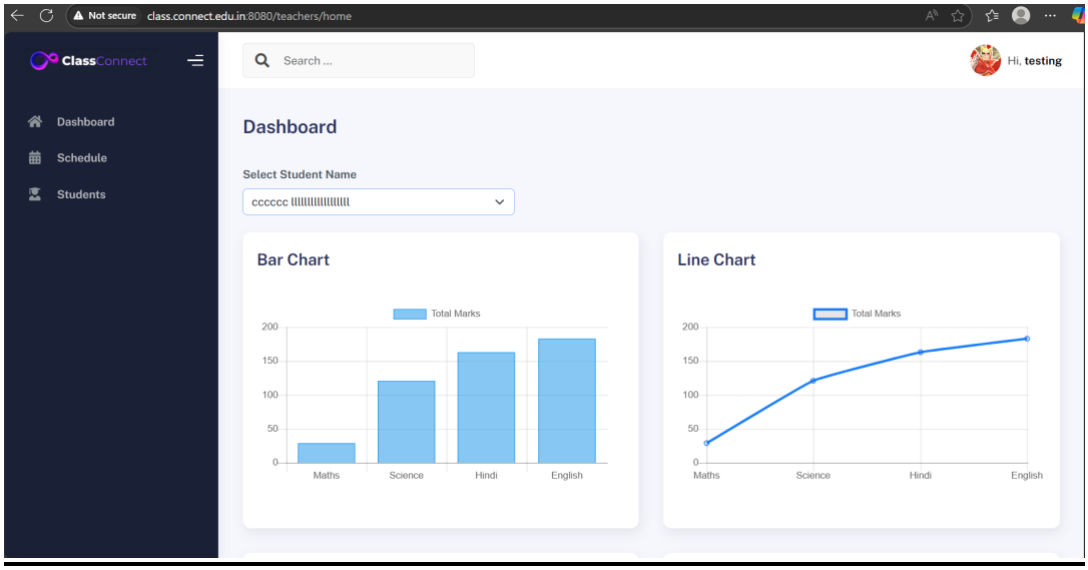


The Teacher Login module provides a secure login interface for teachers to access their dashboard and functionalities. The frontend login page allows teachers to log in using either their email or username along with their password. It also includes an option to stay logged in and a “Forgot password” link for password recovery.

On the backend, this functionality is managed by the LoginController class. The /teachers/login GET endpoint displays the login page, while the /teachers/signup GET endpoint serves the teacher signup page. The controller uses the LoginService for authentication logic (implementation not shown here). When teachers enter their credentials, their session is established to maintain login state across pages.

This module ensures that only authenticated teachers can access teacher-specific features, maintaining security and role-based access within the ClassConnect web application.

Teacher Dashboard Module:

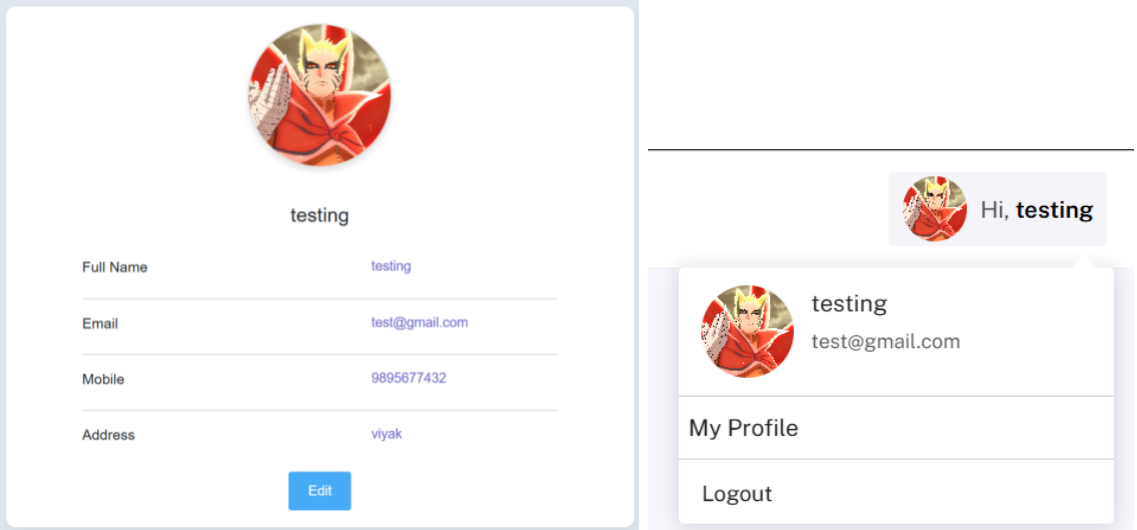


The Teacher Dashboard provides an intuitive interface for teachers to monitor and analyze student performance. After login, teachers can view a dropdown list of their assigned students. Upon selecting any student, their marks are visualized through Bar and Line Charts, enabling quick academic analysis.

On the backend, this functionality is managed by the DashboardController. The /teachers/home GET endpoint fetches all students linked to the logged-in teacher by retrieving the teacher_id from the session. The students list is added to the model and displayed on the dashboard page. Another endpoint, /teachers/home/{studentId}, returns the selected student's marks in JSON format using @ResponseBody. The FetchStudentsService service is used to fetch both student lists and individual student marks.

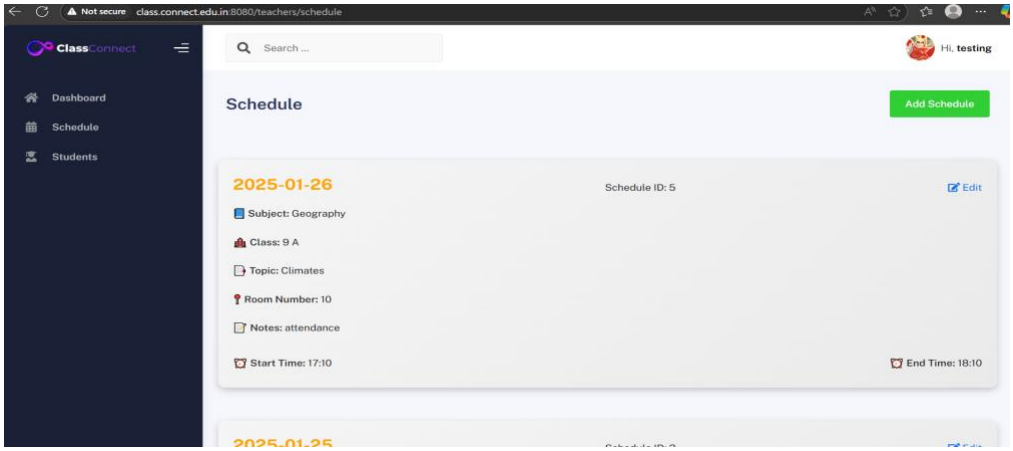
Overall, this module integrates dynamic data fetching and chart visualization, helping teachers easily track student performance within the ClassConnect application.

Teacher Profile Module:



The Teacher Profile Module enables individual teachers to view and update their personal profile details within the ClassConnect application. This module is managed by the TeacherProfileController, which handles requests for viewing, editing, and updating teacher profiles. Teachers can access their profile information, including full name, email, mobile number, address, and profile image, via the /teachers/profile endpoint. The edit functionality is provided through /teachers/editProfile, where teachers can modify their details and upload a new profile image if desired.

The updated information is then saved using the /teachers/updateProfile POST endpoint. Uploaded images are stored in a structured directory under uploads/images/{teacherId}/, ensuring organized file management. The controller leverages the ProfileService for database interactions, while maintaining user session integrity by retrieving the teacher_id from the HTTP session. Overall, this module ensures that each teacher maintains an up-to-date and personalized profile within the system, enhancing usability, professionalism, and data accuracy in the application.



This controller is part of the teachersController package and is responsible for handling the posting of daily schedules by teachers. It uses ScheduleService to perform business logic related to schedules. The controller defines a method postScheduleForm mapped to POST /teachers/schedule/post.

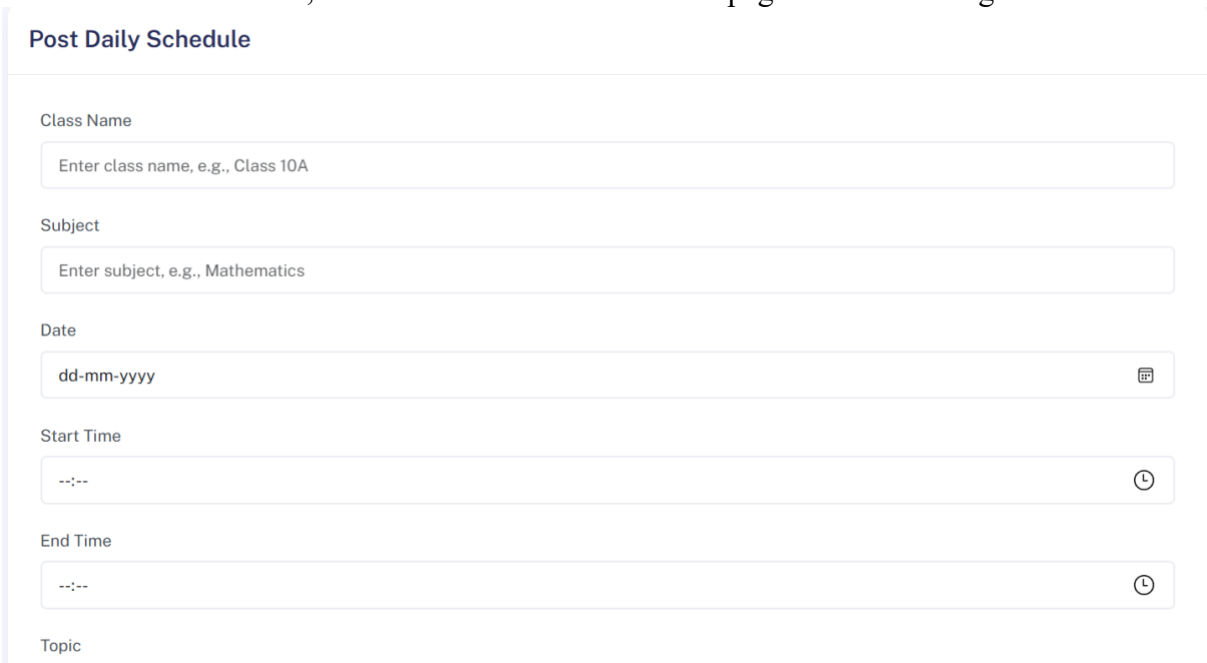
When a teacher submits the schedule form, this method retrieves the teacher_id from the session to ensure that the schedule is associated with the correct teacher. It sets this teacher_id into the ScheduleModel object, then prints the schedule model details to the console for debugging. The method calls scheduleService.createSchedule(scheduleModel) to save the schedule in the database. Finally, it adds a flash attribute message “Data inserted successfully!” to display on redirection and returns a redirect to the schedule dashboard page.

This controller handles all functionalities related to fetching, displaying, and deleting students data for teachers. It uses FetchStudentsService and FetchStudentsRepository to perform student data operations.

The getAllStudents method is mapped to GET /teachers/students. It fetches the teacher_id from the session to retrieve only the students under that teacher. It then calls fetchStudentsService.getAllStudents(teacherId) to get the list of students, prints their image URLs for confirmation, and adds this list to the model with the key "students". The page rendered is teachers/students, displaying the students list.

The getStuendsMarks method is mapped to GET /teachers/add_student or /teachers/add_student/{student_id}. It checks if a student_id is provided. If yes, it converts it to Long and uses fetchStudentsService.getStudentMarks(studentId) to fetch the marks of that student. It also calls fetchStudentsService.getStudentById(studentId) to get detailed student data. Both marks and student data are added to the model with relevant attribute names, and the page rendered is teachers/add_student.

The deleteStudent method is mapped to GET /teachers/delete_student/{student_id}. It calls fetchStudentsService.deleteStudent(student_id) to delete the student record from the database. After deletion, it redirects back to the students page to reflect changes.

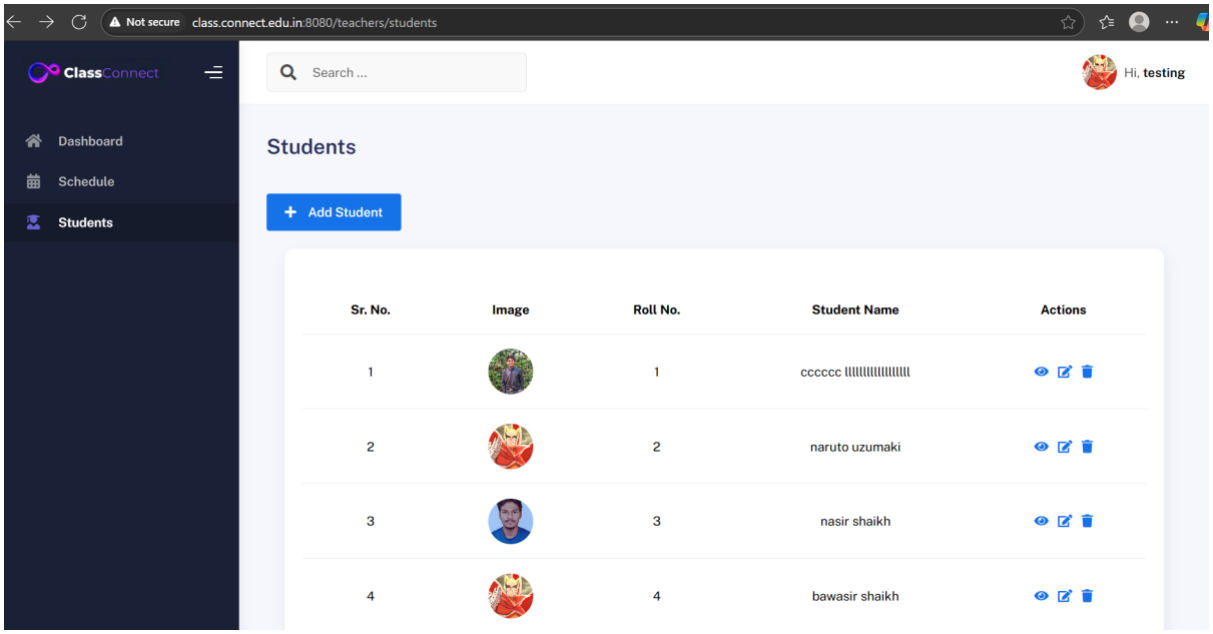




This controller handles the editing and updating of existing schedules. It is under the teachersController package and uses UpdateScheduleService for performing update operations. The controller contains the postScheduleForm method mapped to POST /teachers/schedule/editPost.

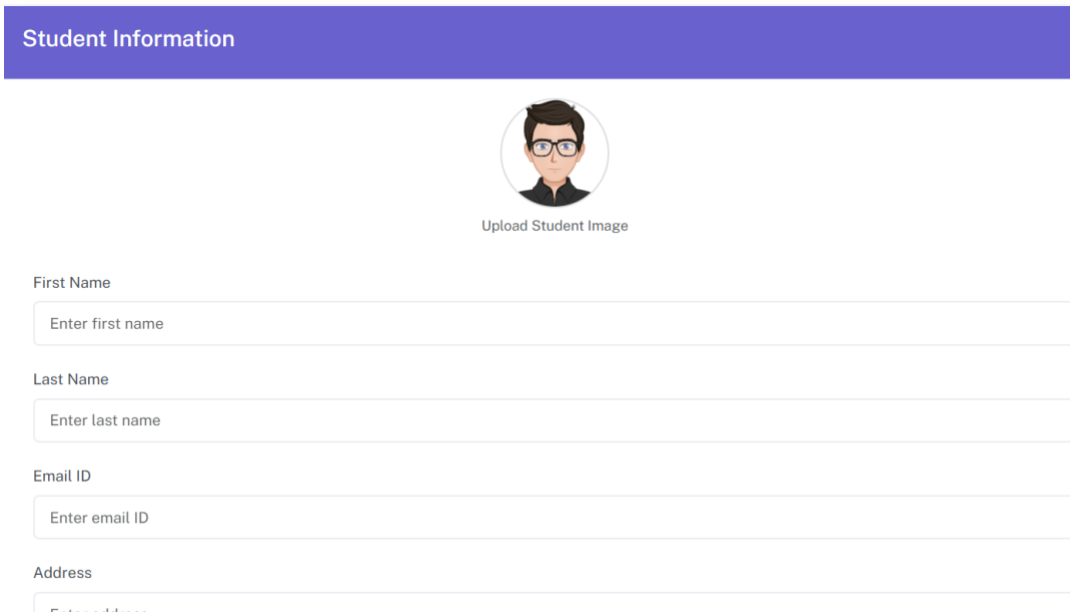
This method receives multiple request parameters such as schedule_id, className, subject, date, startTime, endTime, topic, roomNumber, and notes. These fields are used to create an UpdateScheduleModel object containing all updated schedule data. The method sets each of these values explicitly into the model, then calls updateScheduleService.updateSchedule(updateScheduleModel) to update the record in the database. A flash attribute with message “Data Updated successfully!” is added to notify success, and the user is redirected back to the schedule dashboard page.

Teacher Add Student Module:



addStudents Method

Mapped to POST /teacher/students, this method adds a new student. It receives first name, last name, email, address, and student image. It first checks if any field is empty and redirects back with an error if so. It then gets the teacher_id from session, generates a unique student ID using current time in milliseconds, saves the image using saveImage(), and generates a username (first name + random 3 digits) and password (part of first name + '@' + random digits).



An AddStudentModel is created with all details including is_active as "0" and saved to the database. It then sends an email with the generated username and password. Finally, it redirects to the students list page with a success message.

2 generateRandomStudentId Method

Generates a unique student ID using the current time in milliseconds.

3 saveImage Method

Saves the uploaded student image in the configured uploads folder and returns its relative path for database storage. Returns null if saving fails.

4 generateUsername Method

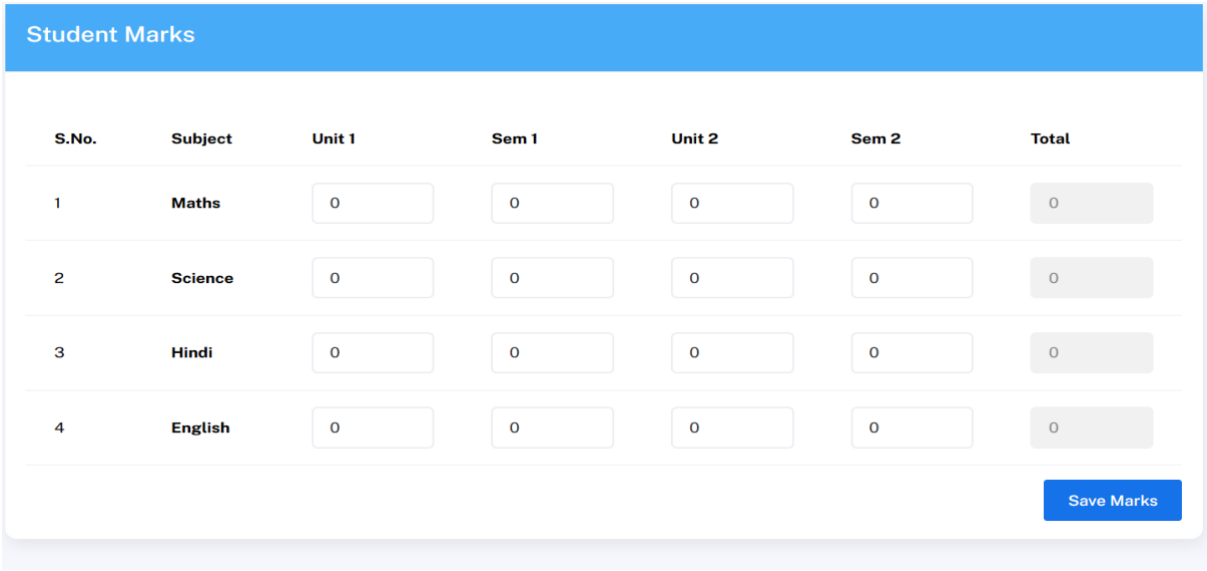
Generates a username by combining the first name (lowercase) and a random 3-digit number.

5 generatePassword Method

Creates a password using up to 5 letters of the first name + '@' + random digits to make it at least 8 characters long.

6 sendEmail Method

Uses JavaMailSender to send an email to the student with their username and password after successful registration.



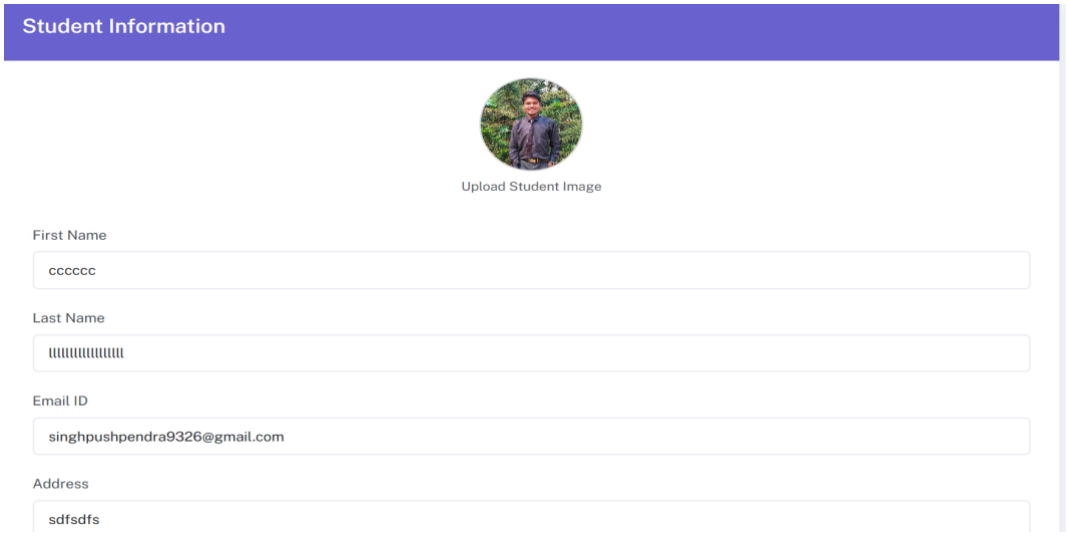
7 saveMarks Method

Mapped to POST /teachers/save_marks, it saves or updates student marks. It receives lists of marks for Unit 1, Unit 2, Semester 1, Semester 2, subjects, optional IDs for updating, and student_id. It loops through subjects, creates an AddStudentMarksModel for each, sets marks and total, and saves them. Finally, it redirects to the students page.

Edit Student Module:

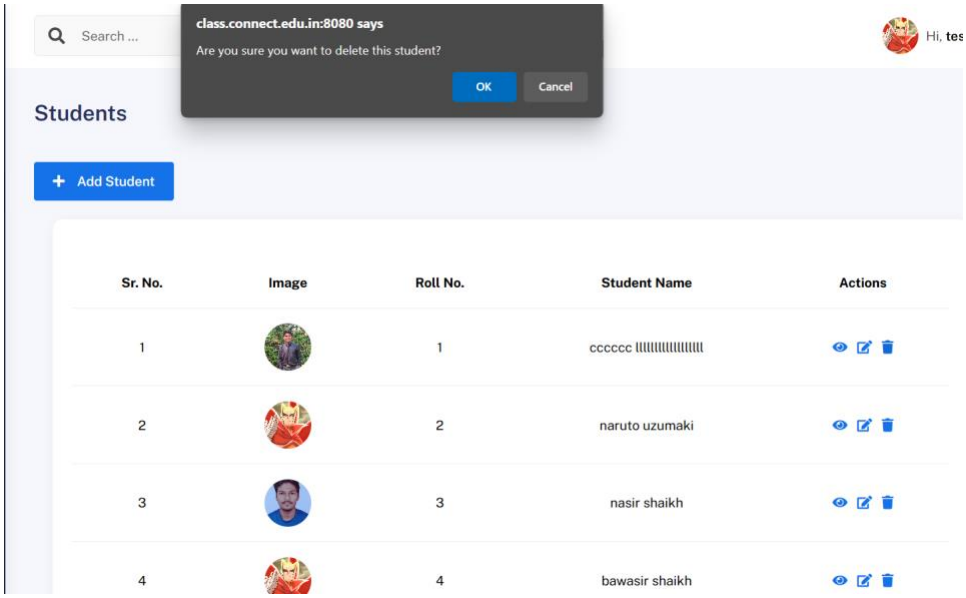
The edit student functionality in the ClassConnect web application allows a teacher to modify the details of an existing student. On the student list page, the edit action is triggered by clicking the pencil icon located in the “Actions” column beside the student’s name. This

leads to an edit form that is pre-filled with the student's existing information such as first name, last name, email ID, address, and profile image, making it convenient for the teacher to update only the necessary fields. Additionally, teachers have the option to upload a new image by clicking on the "Upload Student Image" section. After making the desired changes, the teacher submits the form, which sends the updated data to the backend. The backend processes the request, performs necessary validations, and updates the student record in the database. Upon successful update, the system redirects the user back to the student list page where the revised student details are displayed.



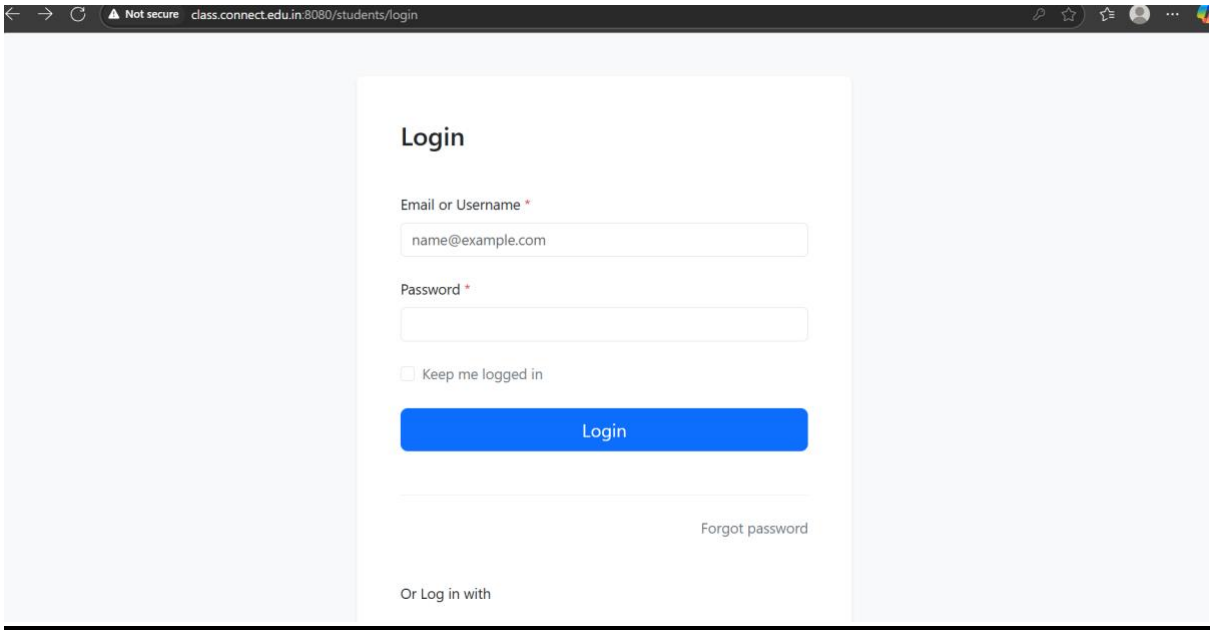
Delete Student Module:

The delete student functionality enables a teacher to permanently remove a student's record from the system. This operation is initiated by clicking the trash icon in the "Actions" column of the student list page. When the icon is clicked, a browser confirmation dialog appears with the message: "Are you sure you want to delete this student?"—providing a safeguard against accidental deletions. If confirmed, a GET request is sent to the endpoint /teachers/delete_student/{student_id}, where {student_id} is the unique identifier of the student to be removed. The corresponding controller method then invokes fetchStudentsService.deleteStudent(student_id) in the service layer, which performs the actual deletion from the database. Once the deletion is complete, the system redirects the user back to the student list page, now updated to exclude the deleted student. However, it is important to note that using a GET request for deletion is not secure or REST-compliant; replacing it with a POST or DELETE request is recommended to enhance security.



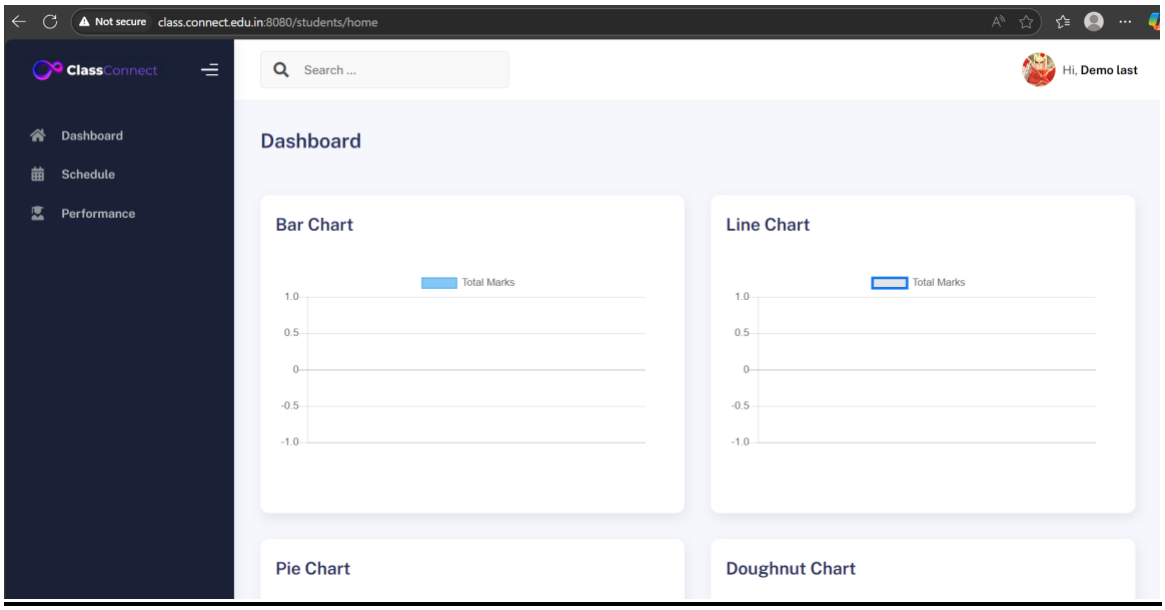
Student Module:

Student Login Module:



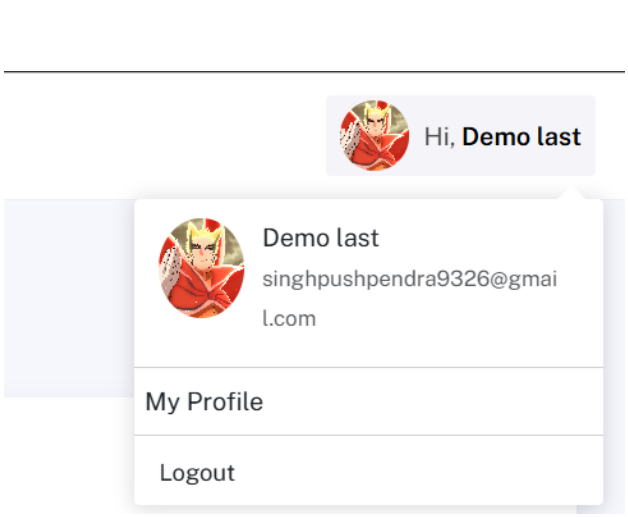
The student login page provides a secure entry point for students to access their personalized dashboard within the ClassConnect system. The login interface includes fields for either an email or username and a password, along with a "Keep me logged in" checkbox to maintain the session. Upon submitting valid credentials, students are authenticated and redirected to the dashboard. The login route is mapped to /students/login in the StudentsAppController, which returns the students/login.html view from the templates folder. The login page also features a "Forgot password" option, offering a recovery path in case students lose access to their credentials. This page is designed to be simple and user-friendly to streamline the login process for all users.

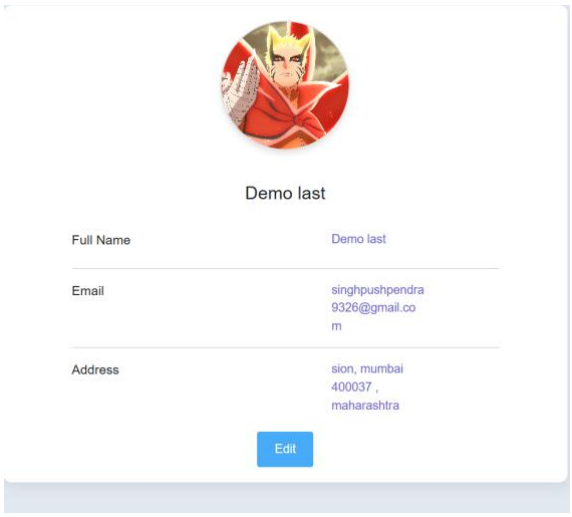
Student Dashboard Module:



Once logged in, students are redirected to the dashboard, accessible via /students/home. This view, handled by the index() method in the StudentsAppController, loads the students/index.html template. The dashboard provides a comprehensive overview of the student's academic status through visual charts such as bar, line, pie, and doughnut charts—each representing different aspects of performance, such as total marks or subject-wise analytics. A navigation sidebar on the left allows students to easily switch between key sections: Dashboard, Schedule, and Performance. These sections are served by additional controller methods (/students/schedule and /students/performance) which return their respective views. The clean layout and chart integration help students quickly understand their progress, making the dashboard an essential component of the student experience in ClassConnect.

Student Profile Module:





Student Profile – View and Edit

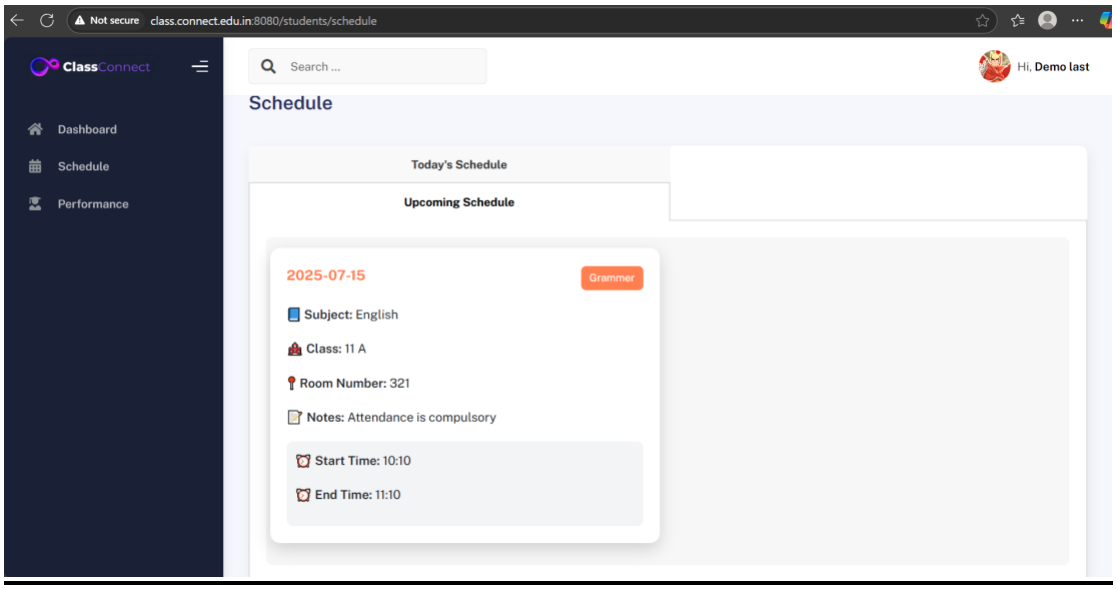
The student profile module allows a logged-in student to view and manage their personal information, including their full name, email, address, and profile picture. Upon accessing the profile from the dashboard header (by clicking the profile icon), students are redirected to the profile view page that displays all details in a read-only format, along with an "Edit" button.

Clicking the “Edit” button navigates the student to the profile editing interface. This edit page presents a form pre-filled with existing student data such as first name, last name, email, address, and current profile image. Students can update these fields and optionally upload a new profile image. This setup ensures a seamless user experience while updating only the necessary data.

When the form is submitted, the updated profile information is sent to the backend using a POST request to /students/updateProfile. If a new image is uploaded, it is saved in a specific directory structure (uploads/images/students/{studentId}/) using the configured upload path. If no new image is provided, the system retains the existing profile image. The backend handles file saving using the MultipartFile mechanism, and proper directory creation is ensured before saving the image.

On successful update, the student is redirected back to the profile view page to see the latest changes. In case of any issues (such as image upload failures), a generic error page is displayed with the corresponding message. This module relies on HttpSession to retrieve the currently logged-in student's ID, ensuring data consistency and access control per session.

Student Schedule Module:



The student schedule module provides students with an organized view of their class timetable, split into Today’s Schedule and Upcoming Schedule tabs. This allows students to easily identify their current day’s responsibilities and prepare for future classes.

When a student navigates to the Schedule page (/students/schedule), the backend controller StudentScheduleController is triggered. This controller uses the StudentScheduleService to fetch two lists of scheduled classes: one for the current day and another for upcoming days.

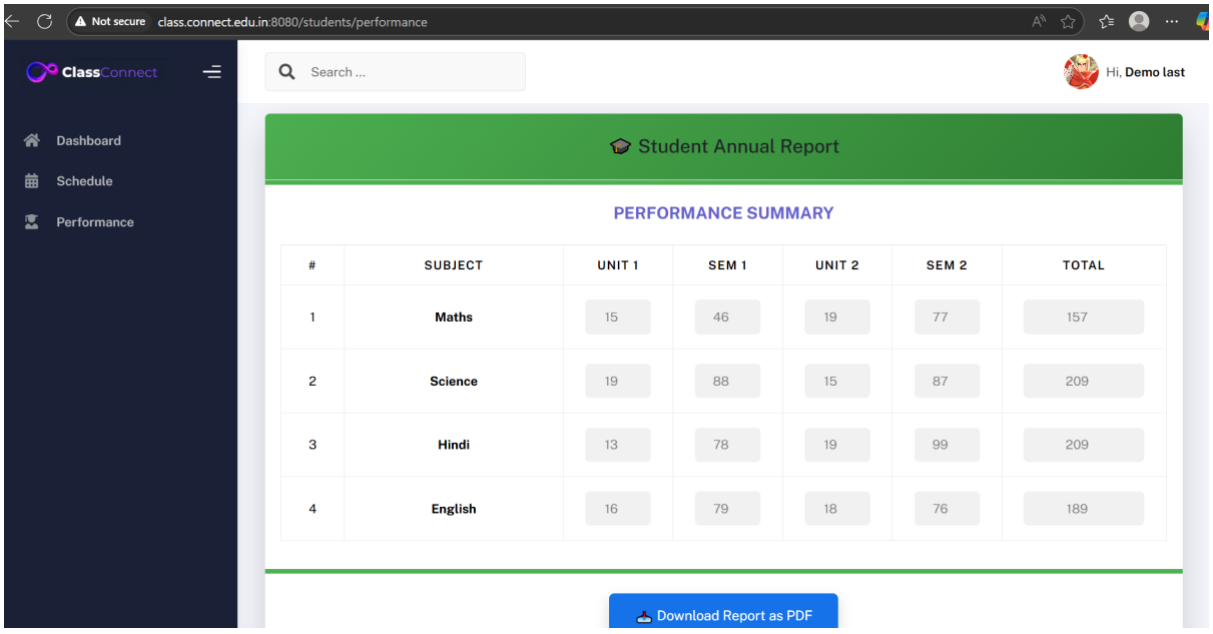
These are retrieved using the getTodaySchedules() and getUpcomingSchedules() service methods, respectively.

The controller obtains the currently logged-in teacher's ID (teacher_id) from the session. Using this ID, it queries for the relevant schedules tied to that teacher. This indicates that the student schedule might be populated based on teacher availability or assignments.

Each schedule entry displayed on the frontend contains rich detail such as subject name, class (e.g., 11 A), room number, timing (start and end time), and important notes like "Attendance is compulsory." A colored label (e.g., "Grammer") highlights the type of session or topic being covered. The clean card-based design enhances clarity and visual separation between different schedule entries.

Finally, all the data is passed to the view template (students/schedule.html), where it is rendered using Thymeleaf. This provides a responsive and dynamic UI for students to track their class schedule with minimal effort.

Student Performance Module (Annual Academic Report):



The Student Performance module provides a clear and structured summary of a student's academic results throughout the year. It is accessible via the /students/performance route, which loads the students/performance.html Thymeleaf template.

On the frontend, the module is titled "Student Annual Report" and is prominently displayed with a green header for visual distinction. Below this, a Performance Summary table is rendered, breaking down the scores across various subjects and evaluations.

Each row in the table corresponds to a subject (e.g., Maths, Science, Hindi, English), and the columns show scores for Unit 1, Semester 1, Unit 2, Semester 2, and a calculated Total. This breakdown allows students to track their progress across different academic phases and identify their strengths or areas needing improvement.

The backend controller for this module is extremely minimal, using only a @RequestMapping("/students/performance") that directly returns the template name. This implies that the data shown in the table is either hardcoded, fetched client-side, or injected into the HTML via model attributes in a different layer (possibly through another service or controller enhancement in production).

At the bottom of the module, there is a "Download Report as PDF" button, allowing students to export their performance summary for offline use or sharing with parents and teachers. This improves usability and accessibility.